



Ship Fast, Sleep Well

Fearless Spring Boot Deployments in the AI Era

DATEV Coding Festival 2026 · 6. Oktober 2026



Freitag, 3 Uhr morgens. Du hast Bereitschaft. Alle Endpunkte geben 5xx zurück

**Vibe-codierter
Hotfix**

die KI sagt: sieht gut aus

45 Minuten Pipeline

für eine Ein-Zeilen-Änderung

**Tests, denen keiner
traut**

was verifizieren sie?

Kein Feature Flag

Rollback oder nichts

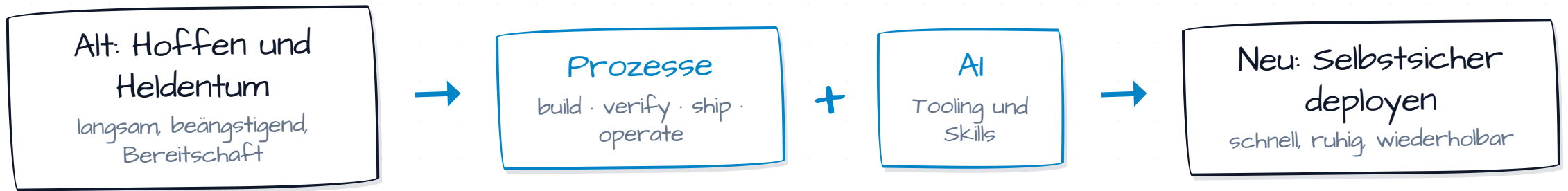
Logs ohne Kontext

welcher User? welcher Request?

Kein Runbook

wer weiß, was zu tun ist?

Unsere Agenda für heute: von Alt nach Neu



KI macht uns schneller. Prozesse machen uns sicher.

Über Philip

- Software Engineer aus Herzogenaurach, direkt neben Nürnberg 🍺
- Fokus auf **Testing** von Java- und Spring-Boot-Anwendungen, Blog und Content 🌱
- Nicht zum ersten Mal bei DATEV: **DATEV Coding Festival 2025** mit einem ganztägigen Workshop zu Spring Boot Testing, und vor zwei Wochen die **DATEV Software Craft Community**, ebenfalls zu Spring Boot Testing



Was dich erwartet: Best Practices aus der Praxis

Was ich selbst in **10 Jahren** in mehreren Teams gesehen habe

Auch aus eigener Erfahrung **on call** bei ImmoScout24, für den zentralen Service mit allen Listings

Nimm mit, was zu deinem Team passt

Wie sieht es bei euch im Team aus?

Geh auf menti.com und gib den Code **1354 9993** ein.

Es gibt **drei Fragen**, bitte beantworte sie.

Wir schauen uns die Ergebnisse gleich **live** an.



Drei Teile des Softwareentwicklungszyklus

1 • Build

Aussagekräftige, schnelle
Testsuite
Context caching
Parallelisierung
Testcontainers
Mutation testing
Statische Checks

2 • Ship

Schnelle Deployments
Rolling Update
Feature flags
Kill Switch

3 • Operate

Structured logging + MDC
Distributed tracing
Alerting
Runbooks
Canary tests

KI und Skills über alle drei Teile

01 - BUILD

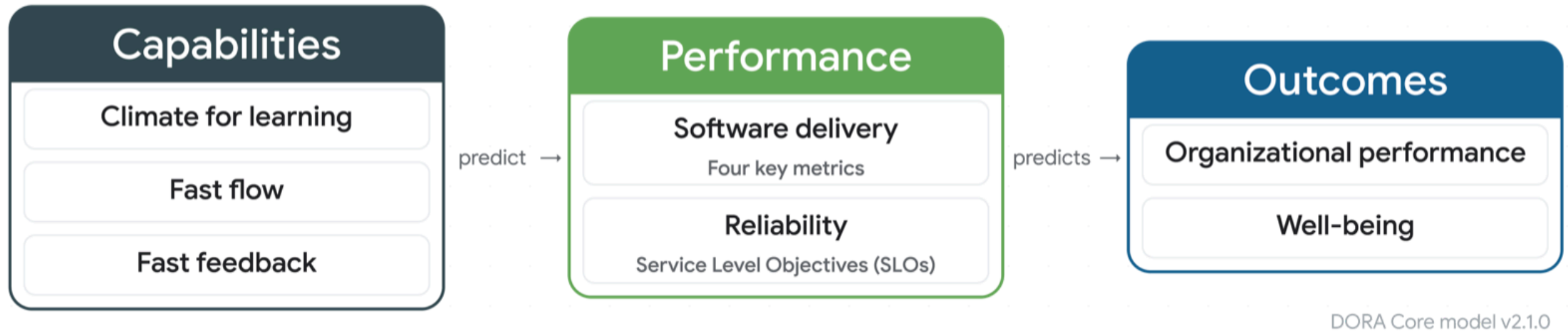
Eine aussagekräftige, schnelle Testsuite

**Die Testsuite ist dein
Sicherheitsnetz. Sie muss schnell
und zuverlässig sein.**

Belegt durch die DORA-Forschung

DORA Core Model

dora.dev/research



Das DORA Core Model stellt **Fast Feedback** neben Fast Flow und ein Klima des Lernens.

- Diese Capabilities sagen die **Software Delivery Performance** voraus
- Delivery Performance sagt die **Organizational Performance** und das Wohlbefinden voraus

Schnell und zuverlässig

Schnell

Parallelisierung

Context caching

Testcontainers-Optimierungen

Zuverlässig

Braucht **Wissen** und **Judgment**

Was testen, welcher Slice, keine flaky Tests

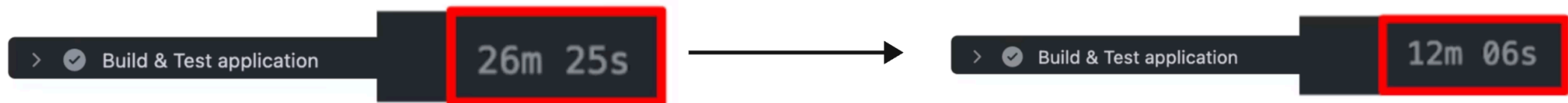
Mutation testing

Best Practices werden zu **Skills**

Das Spring-Test-Hidden-Gem Nr. 1

- Den `ApplicationContext` zu starten kostet Testlaufzeit
- Jeder Context-Start (Slice oder voll) dauert mehrere Sekunden
- Spring Test löst das mit: **TestContext Context Caching**

Ergebnisse bei einem meiner Kunden:



Context Caching in a Nutshell

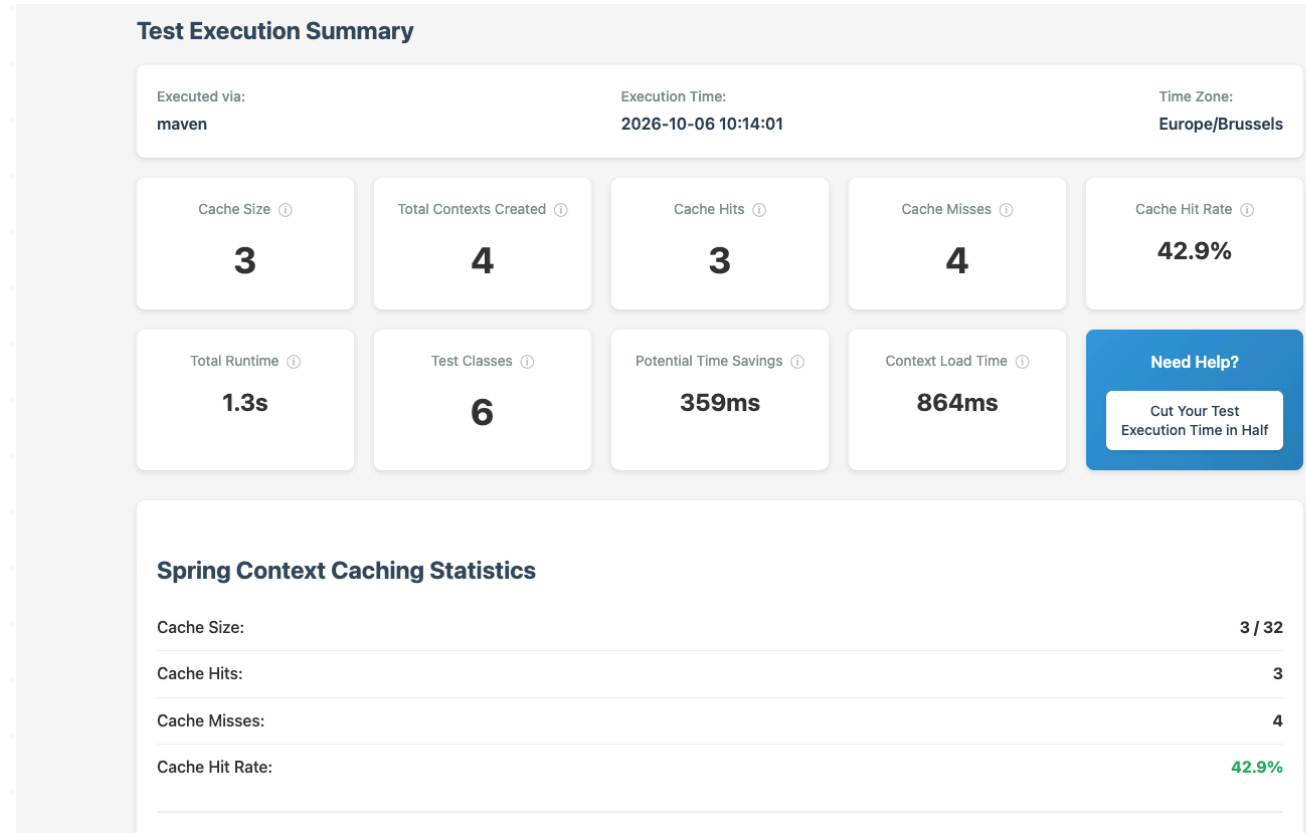
```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LinkedHashMap<>(32, 0.75f, true));
```

- Spring Test baut aus Profilen, Properties, Klassen usw. eine eindeutige `ApplicationContext`-Konfiguration (`MergedContextConfiguration`)
- Braucht ein späterer Test exakt dieselbe Konfiguration, übergibt Spring einen "heißen" Context
- Ziel: so wenige Konfigurationsvarianten wie möglich, so viele Cache-Treffer wie möglich

Spring Test Profiler



- Open-Source-Tool: zeigt, **wie viele Contexts** deine Testsuite startet
- **Cache Hits, Misses** und die Zeit für Context-Starts
- Findet die Ursache: Properties, `@MockitoBean`, `@DirtiesContext`



Parallele Tests und schnellere Container

Testparallelisierung

Thread-Pool-basierte Parallelisierung mit JUnit oder JVM-Forks mit dem Build Tool

Testcontainers

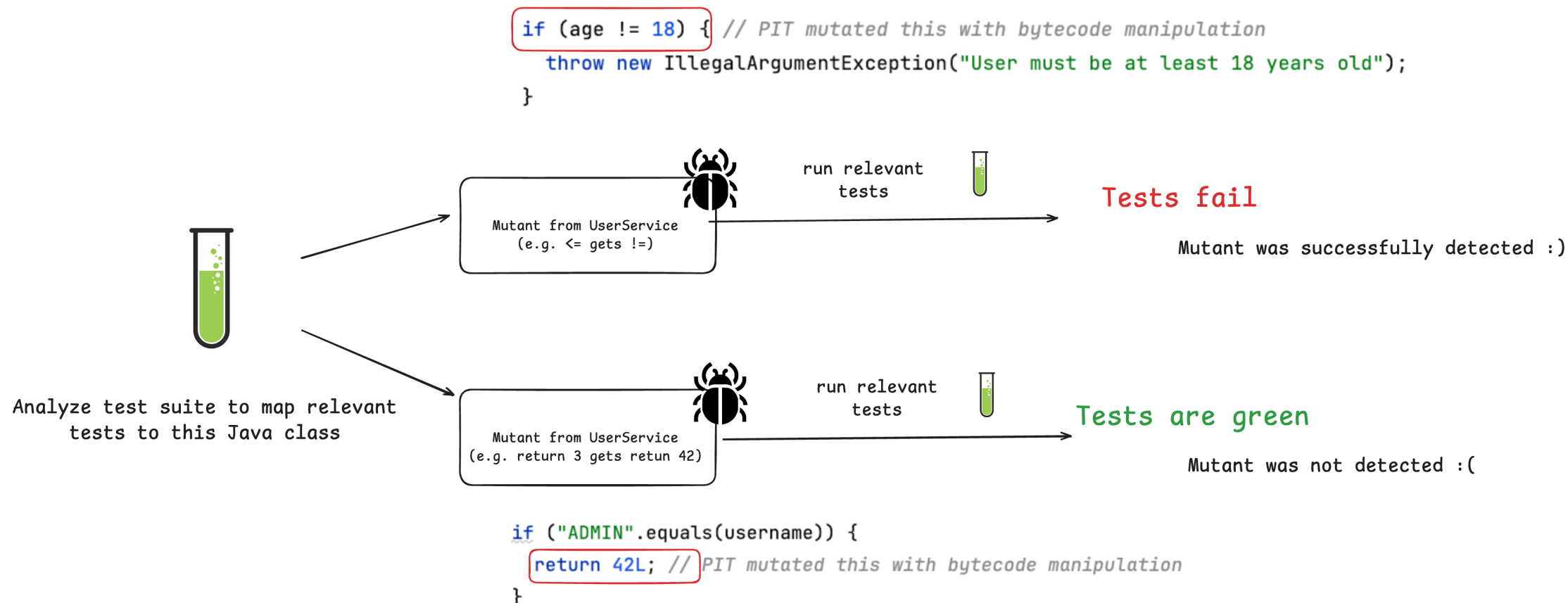
möglichst wenig Container-Starts,
Wiederverwendung

Challenge: Code Coverage

Stell dir Unit Tests für diese isolierte Business-Logik vor:

```
1  public Long registerUser(int age, String username) {  
2  
3      if (age <= 18) {  
4          throw new IllegalArgumentException("User must be at least 18 years old");  
5      }  
6  
7      if ("ADMIN".equalsIgnoreCase(username)) {  
8          throw new IllegalArgumentException("Username 'ADMIN' is not allowed");  
9      }  
10  
11     // ...  
12  
13 }
```

Idee: Regressionen einbauen, um die Testqualität zu prüfen



Meine Skill-Library für Spring Boot Testing

```
1  .claude/skills/spring-boot-testing/  
2  |— unit-testing           schnelle Tests ohne Context-Ballast  
3  |— slice-testing         passend große Spring-Context-Slices  
4  |— slice-web-testing     Web-Schicht mit @WebMvcTest  
5  |— integration-testing   Full-Context-Tests, die schnell bleiben  
6  |— testcontainers-setup  echte Infrastruktur, ein Container pro Image  
7  |— e2e-ui-testing        User Journeys gegen die laufende App  
8  |— test-setup-review     erkennt Test-Anti-Patterns für dich
```

Regeln, Best Practices und Anti-Patterns, **einmal** aufgeschrieben. Der Agent folgt **deinem** Judgment.

Beispiel: unit-testing Skill und testing-standards.md

```
1 unit-testing/  
2   └─ SKILL.md           wann nutzen, Scope, Non-negotiables  
3   └─ references/  
4       └─ testing-standards.md   Regelkatalog mit ID und Severity
```

```
1 [T1-C]   Every behavior change ships with a unit test  
2 [T2-H]   One assertion per test  
3 [T7-H]   Every assertion carries a failure message  
4 [T9-H]   No @BeforeEach and no test-class fields  
5 [T27-H]  Every wait has a timeout, never Thread.sleep
```

Jede Regel hat eine **ID** und eine **Severity** (C, H, M, L). Der Agent nennt sie in jedem Befund.

Jeden möglichen Check automatisieren

SonarQube

Codequalität und Security Hotspots

Statische Analyse

Error Prone · SpotBugs · ArchUnit

Formatting

Spotless · Checkstyle

Abhängigkeiten

Renovate · Dependabot

Secret Scanning

keine Keys in Git

Quality Gates

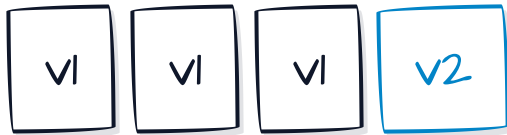
klare Regeln, wann ein Build
abgebrochen werden soll

02 - SHIP

Deployment $\neq$ Release

Schnelle Deployments: Rolling Update

1. v2 starten



Neuer Container startet,
Health Check wird grün

2. Umschalten



Traffic wechselt, ein alter
Container stoppt

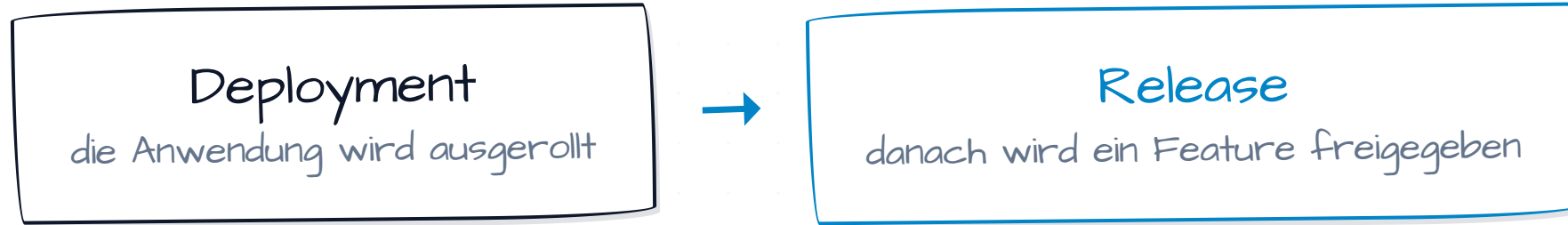
3. Wiederholen



Alle 2-3 Container laufen
mit der neuen Version

Der gesamte Rollout in ~5-8 Minuten. Je schneller, desto besser.

Deployment vom Release entkoppeln



Klein und häufig deployen. Veröffentlichen, wenn du bereit bist.

Feature Flags mit Toggly

```
1  public enum Features implements Feature {  
2  
3      @Label("New checkout flow")  
4      NEW_CHECKOUT;  
5  
6      public boolean isActive() {  
7          return FeatureContext.getFeatureManager().isActive(this);  
8      }  
9  }
```

Die Togglz Admin Console

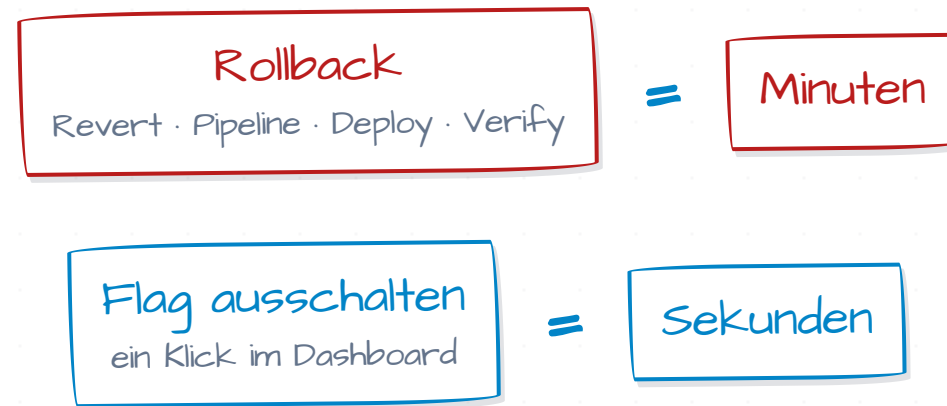
Togglz application

All Features

Feature	Status	Strategy	Actions
New checkout flow ⓘ Owner: team-checkout	Enabled	Users by name Users: anna, ben	
Pay by card Owner: team-payment	Enabled		
Fast product search Owner: team-search	Enabled	Gradual rollout Percentage: 20	

Togglz 4.6.4
www.togglz.org
Apache Tomcat/11.0.24

Kill Switch: schneller als ein Rollback



03 - OPERATE

Sehen, finden, beheben

Standard-Log: lesbar, aber schwer abfragbar

```
1 2026-10-06T10:15:32.481+02:00 ERROR 4711 --- [nio-8080-exec-3] d.p.shipfast.CheckoutService : Payment failed
2 2026-10-06T10:15:32.502+02:00  WARN 4711 --- [nio-8080-exec-7] d.p.shipfast.CheckoutService : Retrying payment
3 2026-10-06T10:15:33.114+02:00 ERROR 4711 --- [nio-8080-exec-3] d.p.shipfast.CheckoutService : Payment failed
```

Welcher **User**? Welche **Bestellung**? Welcher **Request**?

Strukturiertes Logging mit MDC: JSON und Extra-Felder

```
1 MDC.put("userId", authentication.getName());
```

```
1 {  
2   "log.level": "ERROR",  
3   "message": "Payment failed",  
4   "userId": "u-4711",  
5   "orderId": "o-815",  
6   "trace.id": "4bf92f3577b34da6"  
7 }
```

Die Log-Abfrage `userId:u-4711` findet jetzt genau das, was dieser User getan hat.

Distributed Tracing: Wo liegt das Problem?

POST /orders

480 ms

OrderService.placeOrder()

465 ms

HTTP POST payment-service (Nachbar-Team)

340 ms

POST /payments (deren Endpunkt)

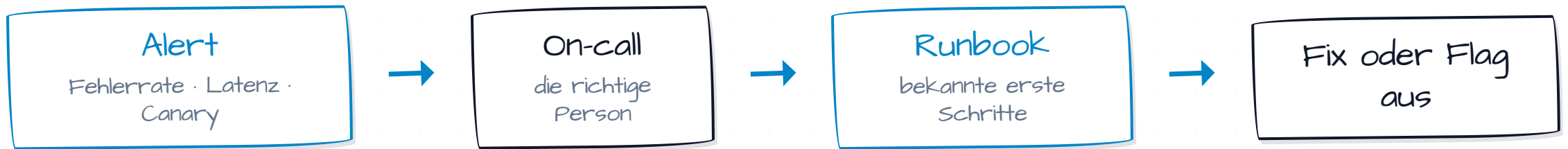
310 ms

INSERT INTO orders (unsere DB)

65 ms

Ein Trace geht über die **Team-Grenze**: der langsame Span liegt im Service des **Nachbar-Teams**, nicht in unserem Code.

Alerting und Runbooks



Ein Runbook: Symptom · Auswirkung · erste Checks mit Deep Links zu Logs, Traces und Dashboards · Gegenmaßnahme · Eskalation

Beispiel: Datadog Alert mit Runbook-Link

```
1 [Triggered] checkout-service: Fehlerrate über 5 %
2
3 {{#is_alert}}
4 Fehlerrate: {{value}} % (Schwellwert: {{threshold}} %)
5
6 Runbook: https://wiki.example.com/runbooks/checkout-5xx
7 Logs:    https://app.datadoghq.eu/logs?query=service:checkout status:error
8
9 @pagerduty-checkout-oncall
10 {{/is_alert}}
```

Der Alert nennt das **Symptom** und führt mit **einem Klick** zum Runbook und zu den Logs.

Beispiel-Runbook: ELB-5xx-Alarm

Bedeutung

User sehen viele
HTTP-5xx-Fehler

Auswirkung

User können nicht
mit der App
arbeiten

Clients können
nicht
synchronisieren

Diagnose

Laufender
Plattform-
Incident?

Logs

Operations-
Dashboard

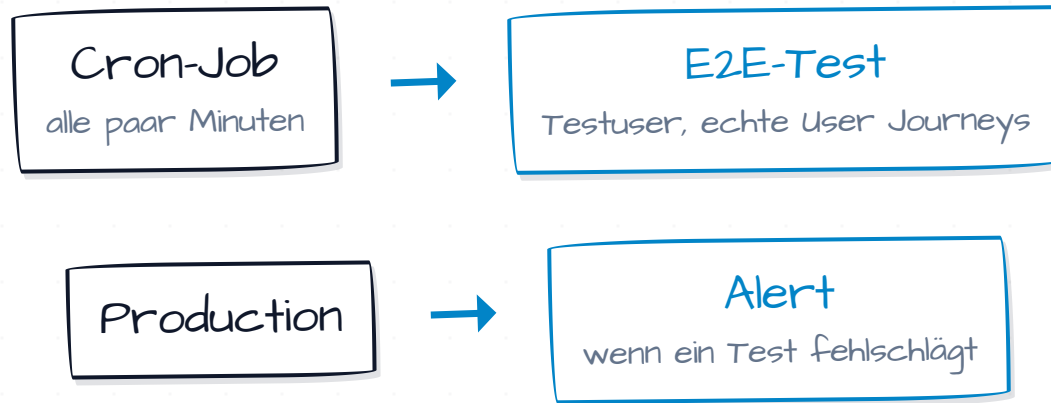
Gegenmaßnahme

Provider down: Feature
abschalten

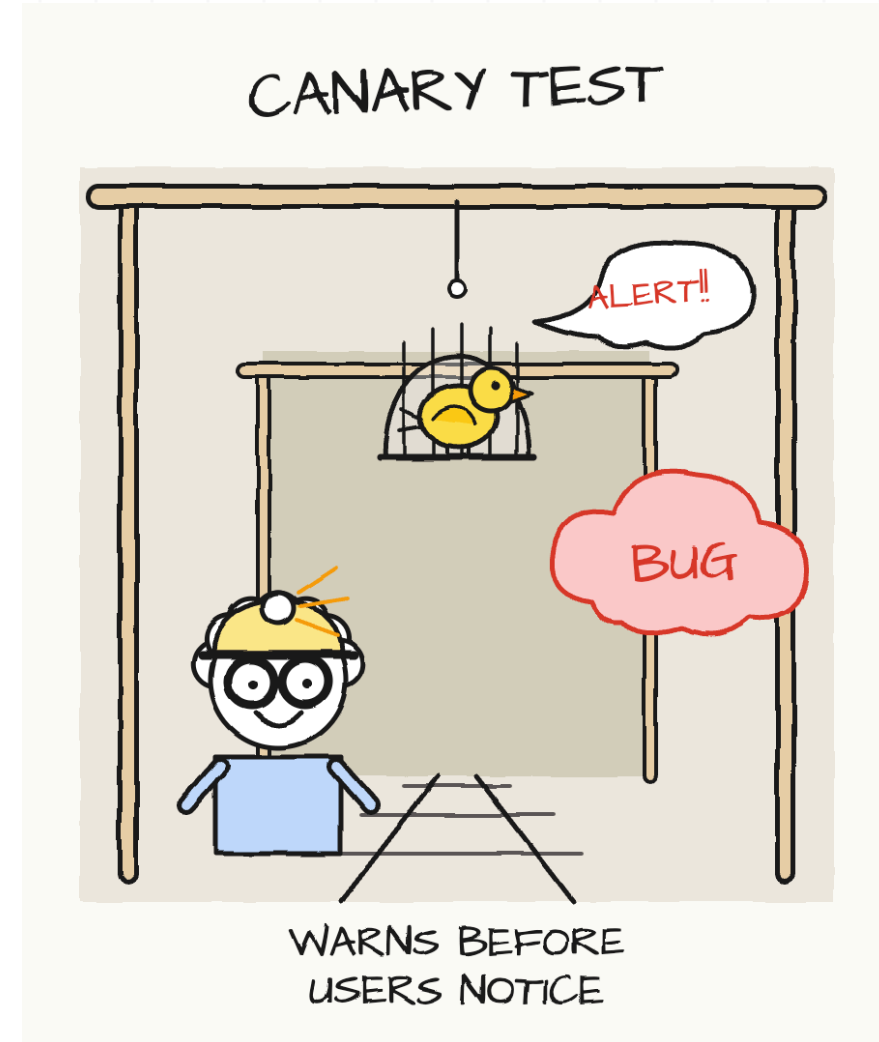
Schlechtes Release: Revert
und Redeploy

Canary Testing in a Nutshell

Cron-basierte E2E-Tests, die regelmäßig laufen



Ziel: Probleme erkennen, bevor der Kunde sie merkt



04 - DIE ROLLE VON KI

KI kontrolliert einsetzen

Skills tragen dein Urteilsvermögen

Test-Skill

was einen sinnvollen Test ausmacht

Struktur-Skill

wie wir Tests schichten und benennen

Logging-Skill

JSON-Logs, MDC, keine Secrets

Review-Skill

erkennt Test-Anti-Patterns

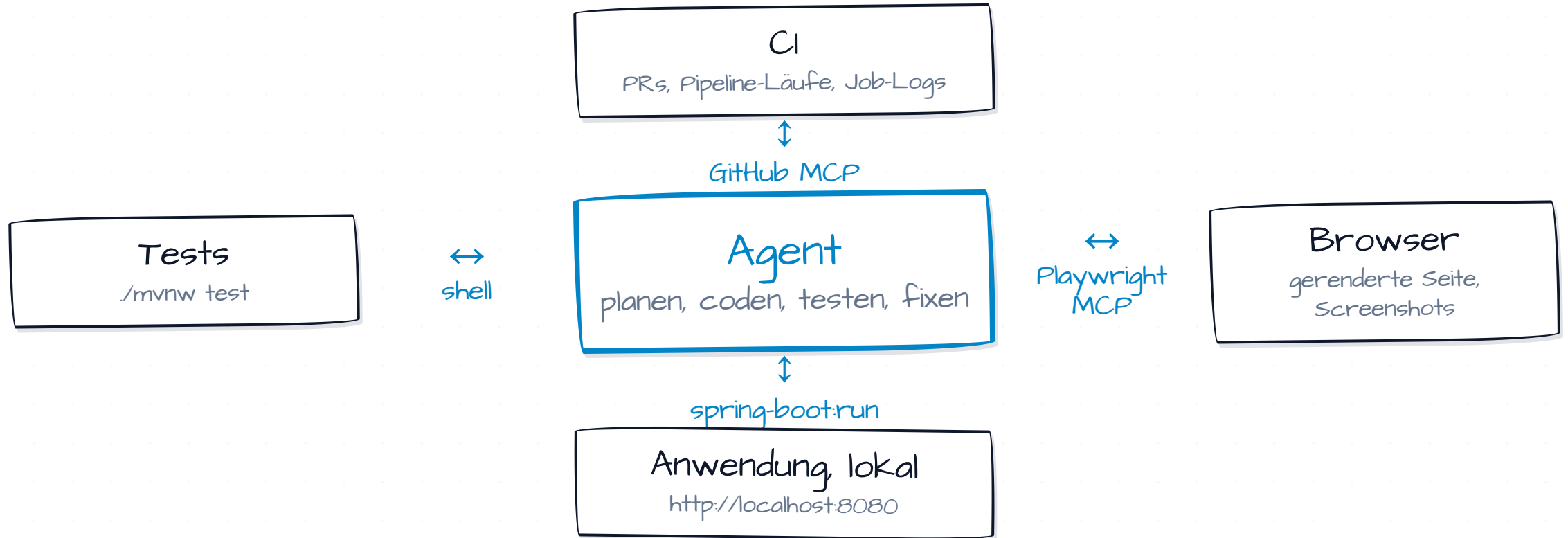
Runbook-Skill

Entwurf aus Alert und Code

Pipeline-Skill

schnell und reproduzierbar halten

KI-Agent-Loop: enger Feedback-Loop mit dem Menschen



Du prüfst jeden Schritt: ein enger Loop, Sekunden statt Tage

Schnelle Tests

aussagekräftig und gecacht

Statische Checks

Sonar · Formatting

CI/CD

schnell und reproduzierbar

Feature flags

Deployment ist kein Release

Logs und Traces

JSON · MDC · Spans

Alerting

die richtigen Leute, schnell

Runbooks

wissen, was zu tun ist

Canary-Tests

echtes Feedback

Fehler lassen sich nicht vermeiden. Viel wichtiger ist, sie früh zu erkennen und im Ernstfall schnell und sicher handeln zu können.

Fearless Shipping!

Der **Spring Boot Testing Newsletter**: Best Practices,
Recipes & Quick Wins direkt in dein Postfach -
rieckpil.de/newsletter



[LinkedIn \(Philip Riecks\)](#)

